

Capítulo 8

Datos Compuestos¹

Hasta el momento hemos visto únicamente cómo operar con valores simples (esto es, con números, con strings, y con valores lógicos). Sin embargo, en computación es recurrente el tener que manipular valores *compuestos* que corresponden a alguna combinación sobre estos valores simples. Por ejemplo, supongamos que queremos calcular la suma entre dos fracciones dadas.

Es claro que una primera alternativa para resolver este problema es definir una función sobre *cuatro* valores enteros, y operar sobre ellos como es usual en matemática:

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd}$$

Así pues, la función que permite resolver este problema es:

```
1 # sumaFracciones: int int int int -> float
2 # calcula la suma entre dos fracciones a/b y c/d
3 # ejemplo: sumaFracciones(1, 2, 3, 4) devuelve 1.25
4 def sumaFracciones(a,b,c,d):
5     return (a * d + b * c) * 1.0 / b * d
6
7 # Test (usa la funcion cerca)
8 epsilon = 0.000001
9 assert cerca(sumaFracciones(1, 2, 3, 4), 1.25, epsilon)
```

Sin embargo, notemos que en el ejemplo anterior, el valor que devuelve la función NO es una fracción, sino un número real que corresponde a la *representación decimal* del resultado. ¿Cómo podemos entonces indicarle al computador que queremos manipular fracciones?

8.1 Estructuras (*structs*)

Una *estructura* (struct) es un tipo de datos que permite encapsular un conjunto fijo de valores (de uno o más tipos), representados por atributos, para conformar un único valor compuesto. En efecto, podemos representar una *fracción* como una estructura formada por dos atributos: un *numerador* y un *denominador*.

¹Parte de este capítulo fue traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

Para trabajar con estructuras en este curso, disponemos del módulo `estructura.py` que contiene las definiciones básicas para poder crear estructuras.

```
1 import estructura
2 estructura.crear("nombre", "atributo1 atributo2 ... atributoN")
```

En el ejemplo anterior, importamos el módulo `estructura` y utilizamos la instrucción `crear` para crear una nueva estructura. Notemos que este comando recibe dos parámetros: el *nombre* de la estructura, y un texto con los distintos *atributos* que la representan, cada uno separado por un espacio simple.

Así, por ejemplo, para crear una estructura que represente a una fracción, proseguimos de la siguiente manera:

```
1 import estructura
2
3 estructura.crear("fraccion", "numerador denominador")
```

Finalmente, notemos que podemos acceder directamente a los distintos atributos que definen a la estructura:

```
1 >>> a = fraccion(1, 2)
2   -> fraccion(numerador=1, denominador=2)
3 >>> a.numerador
4   -> 1
5 >>> a.denominador
6   -> 2
```

En la siguiente sección veremos en detalle cómo diseñar apropiadamente una estructura. Luego, completaremos nuestro ejemplo inicial para ver cómo crear funciones que manipulen estructuras.

8.2 Receta de diseño para estructuras

Recordemos que en el Capítulo 3 vimos cómo escribir la receta de diseño para funciones simples. En el Capítulo 5 la extendimos para el caso en que utilizamos condiciones en la definición de una función, y en el Capítulo 6 vimos el caso de funciones con recursión. Ahora veremos qué es lo que debemos hacer cuando nos vemos enfrentados a definir una función que opera sobre estructuras.

En primer lugar, debemos reconocer *desde el planteamiento del problema* cuáles son las estructuras que vamos a necesitar. En efecto, la regla a seguir es que cada vez que necesitemos operar sobre un conjunto de valores relacionados entre sí, debemos escribir una estructura que los encapsule. Si no utilizamos estructuras, lo más probable es que perdamos rápidamente la pista de qué valor pertenece a qué elemento del programa, especialmente cuando tenemos funciones grandes que procesan mucha información.

En segundo lugar, se deben usar estructuras para organizar las funciones. Para ello, utilizaremos una *plantilla* que la acoplaremos a la receta de diseño que ya conocemos. De esta manera, debemos asegurarnos en todo momento que la definición de la estructura efectivamente se corresponde con la plantilla. Los pasos a incorporar en la nueva receta de diseño son:

8.2.1 Diseñar estructuras

Antes de poder desarrollar una función, debemos entender cómo representar la información dada en el problema. Para lograr esto, buscamos descripciones de conjuntos de valores interrelacionados relevantes, y por cada una de ellas, diseñamos una estructura.

8.2.2 Plantilla

Una función que opera sobre datos compuestos, por lo general opera sobre las componentes de las estructuras que recibe como entrada. Para poder recordar claramente cuáles son estas componentes, debemos diseñar una plantilla. Así, una plantilla es un encabezado y un cuerpo de función que lista todas las posibles combinaciones de expresiones que se pueden generar con las entradas de la función. En otras palabras, una plantilla expresa lo que sabemos sobre las entradas, pero aún no nos dice nada sobre cómo va a ser la salida de la función. Luego, utilizamos la plantilla para cualquier función que consuma los mismos tipos de parámetros.

8.2.3 Cuerpo de la función

La plantilla nos debería dar todas las pistas que necesitamos para escribir apropiadamente la función. Al igual que las funciones que hemos estado desarrollando hasta ahora, el objetivo de esta etapa es formular una expresión que compute la respuesta pedida a partir de la información disponible utilizando otras funciones, o bien, las operaciones primitivas del lenguaje de programación. La plantilla nos recuerda cuáles son los atributos disponibles de los parámetros compuestos, y cómo los podemos relacionar entre ellos. Finalmente, escribamos siguiendo la receta de diseño la función que dadas dos fracciones retorne la suma de ellas:

Contenido del archivo fraccion.py

```
1  import estructura
2
3
4  # Diseño de la estructura
5  # fraccion: numerador (int) denominador (int)
6  estructura.crear("fraccion", "numerador denominador")
7
8  # Contrato
9  # sumaFracciones: fraccion fraccion -> fraccion
10
11 # Proposito
12 # crear una nueva fraccion que corresponda a la suma
13 # de dos fracciones f1 y f2
14
15 # Ejemplo:
16 # sumaFracciones(fraccion(1, 2), fraccion(3, 4))
17 # devuelve fraccion(10, 8)
18
19
20 # Plantilla
21 # def funcionConFracciones(fraccion1, fraccion2):
22 #     ... fraccion1.numerador ... fraccion2.numerador ...
23 #     ... fraccion1.numerador ... fraccion2.denominador ...
```

Contenido del archivo fraccion.py (cont)

```
24 #         ... fraccion1.denominador ... fraccion2.numerador ...
25 #         ... fraccion1.denominador ... fraccion2.denominador ...
26
27
28 # Cuerpo de la funcion
29 def sumaFracciones(f1, f2):
30     num = f1.numerador * f2.denominador \
31         + f1.denominador * f2.numerador
32     den = f1.denominador * f2.denominador
33     return fraccion(num, den)
34
35 # Tests
36 f12 = fraccion(1, 2)
37 f34 = fraccion(3, 4)
38 assert sumaFracciones(f12, f34) == fraccion(10, 8)
```

Capítulo 9

Estructuras de Datos Recursivas¹

Tal como lo vimos en el capítulo anterior, una de las maneras para representar información compuesta es usando estructuras. En efecto, las estructuras son útiles cuando sabemos cuántos datos queremos combinar. Sin embargo, en muchos otros casos, no sabemos cuántas cosas queremos enumerar, y entonces formamos una *lista*. Una lista puede tener un largo arbitrario, esto es, contiene una cantidad finita pero indeterminada de elementos.

9.1 Listas

Para el manejo de listas, ocuparemos el módulo `lista.py`, implementado para efectos de este curso. Para poder ocuparlo, primero hay que importar todas sus funciones:

```
1 from lista import *
```

La definición de una lista es recursiva. Por una parte, una lista puede ser *vacía*, es decir no contiene ningún elemento. Esto se obtiene mediante el identificador `listaVacía`, provisto en el módulo `lista`. Por otra parte, una lista puede estar compuesta por elementos que contienen un *valor* y un enlace al resto de la lista. Con esto, es posible crear una lista más larga concatenando otras listas. En efecto, el construir una lista lo podemos entender como ir armando una cadena, uniendo sus eslabones uno por uno.

Así, en una lista distinguimos dos campos en su estructura: el *valor* y la lista *siguiente*. El campo *valor* puede ser de cualquier tipo (básico o compuesto), mientras que el campo *siguiente* es precisamente una lista, tal como los eslabones de una cadena. La definición de la estructura para listas está incluida en el módulo `lista.py`, note en particular en el contrato de la estructura su definición recursiva:

```
1 # Diseño de la estructura
2 # lista : valor (cualquier tipo) siguiente (lista)
3 estructura.crear("lista", "valor siguiente")
```

Para crear una lista nueva, el módulo provee la función `crearLista` que recibe dos parámetros: el *valor* del primer elemento de la lista y el *resto* de la lista. Veamos un ejemplo: supongamos que queremos formar una lista con los planetas del sistema solar. Primero comenzamos con un eslabón de la lista que sólo contiene a Mercurio:

¹Traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

```
crearLista("Mercurio", listaVacia)
```

Luego viene el planeta Venus:

```
crearLista("Venus", crearLista("Mercurio", listaVacia))
```

A continuación la Tierra, y así sucesivamente:

```
crearLista("Tierra", crearLista("Venus", crearLista("Mercurio", listaVacia)))
```

En toda lista distinguimos dos elementos: la *cabeza* y la *cola*. La cabeza de una lista es el valor que está al frente de la lista (es decir, el primer valor disponible). La cola de una lista es todo lo que va encadenado a la cabeza. Así, en nuestro último ejemplo, la *cabeza* de la lista es el string "Tierra", mientras que la *cola* es la lista formada por el eslabón (Venus, (Mercurio, (listaVacia))). En efecto, estos elementos son proporcionados por el módulo:

```
1 >>> L = crearLista("Tierra", crearLista("Venus",
2   crearLista("Mercurio", listaVacia)))
3 >>> cabeza(L)
4   -> "Tierra"
5 >>> cola(L)
6   -> lista(valor='Venus', siguiente=
7   lista(valor='Mercurio', siguiente=None))
```

En general, una lista no tiene por qué contener valores de un único tipo. Por ejemplo, la siguiente lista es completamente válida (aunque no tiene mucho sentido sin conocer el contexto en el que fue creada).

```
crearLista("Pepito", crearLista(41, crearLista(True, listaVacia)))
```

Así, en esta lista el primer elemento es un string, el segundo es un número, y el último es un booleano. Podríamos pensar que esta lista corresponde a la representación de un registro de personal que tiene el nombre de un empleado, la edad, y si este empleado tiene aún días disponibles para tomarse vacaciones.

Supongamos ahora que nos dan una lista de números. Una de las primeras cosas que nos gustaría hacer es sumar los números de esta lista. Más concretamente, supongamos que estamos únicamente interesados en listas de tres números. Así, una `listaDeTresNumeros` es una lista que se puede crear con la instrucción `crearLista(x, crearLista(y, crearLista(z, listaVacia)))`, donde `x`, `y`, `z` son tres números.

Escribamos el contrato, el propósito, el encabezado y ejemplos para una función que sume estos números siguiendo la receta de diseño:

```
1 # sumaTres: listaDeTresNumeros -> num
2 # suma los tres numeros en unaLista
3 # sumaTres(crearLista(2, crearLista(1, crearLista(0, listaVacia))))
4 # devuelve 3
5 # sumaTres(crearLista(0, crearLista(1, crearLista(0, listaVacia))))
6 # devuelve 1
7 def sumaTres(unaLista):
8     ...
```

Sin embargo, al definir el cuerpo de la función nos topamos con un problema. Una lista de la manera en que la hemos construido es una estructura. Así, deberíamos proveer una plantilla con las distintas alternativas que se pueden elegir para construir las expresiones. Por desgracia, aún no sabemos cómo seleccionar los elementos de una lista. Por otro lado, recordemos que a través de la función `cabeza` podemos acceder al primer elemento de una lista, mientras que la función `cola` nos entrega el resto. Veamos más en detalle qué nos devuelven las combinaciones de estas operaciones:

```
1 >>> cabeza(crearLista(10, listaVacía))
2 -> 10
3 >>> cola(crearLista(10, listaVacía)) #listaVacía
4 ->
5 >>> cabeza(cola(crearLista(10, crearLista(22, listaVacía))))
6 -> 22
```

En efecto, la expresión presentada en el último ejemplo se evalúa paso a paso a través de la siguiente secuencia:

```
1 >>> cabeza(cola(crearLista(10, crearLista(22, listaVacía))))
2 -> cabeza(crearLista(22, listaVacía))
3 -> 22
```

La clave está en considerar que `crearLista(unValor, unaLista)` es un valor *como si fuera de tipo lista*. Así, como siempre, comenzamos con las evaluaciones desde adentro hacia afuera, tal como lo hemos hecho hasta ahora.

Luego, utilizando las funciones `cabeza` y `cola` disponibles en el módulo para manejar listas, podemos escribir la plantilla para `sumaTres`:

```
1 # sumaTres: listaDeTresNumeros -> num
2 # suma los tres numeros en unaLista
3 # def sumaTres(unaLista):
4     ... cabeza(unaLista) ...
5     ... cabeza(cola(unaLista)) ...
6     ... cabeza(cola(cola(unaLista))) ...
```

Notemos que las tres expresiones presentadas en la plantilla nos recuerdan que el argumento de entrada de la función, `unaLista`, efectivamente contiene tres componentes y se pueden extraer con alguna de esas expresiones.

9.2 Definición de datos para listas de largo arbitrario

Supongamos que queremos representar el inventario de una juguetería que vende muñecas, sets de maquillaje, payasos, arcos, flechas, y pelotas. Para construir tal inventario, el dueño de la tienda debería comenzar con una hoja de papel en blanco, y luego ir llenándola con los nombres de los juguetes que tiene en los distintos estantes.

De la sección anterior vimos que representar una lista de juguetes es simple: basta con crear una lista de strings, encadenando los eslabones uno por uno. La secuencia para formar la lista podría ser, por ejemplo:

```
listaVacía
crearLista(pelota, listaVacía)
```

APUNTE DE USO INTERNO

PROHIBIDA SU DISTRIBUCIÓN

```
crearLista(flecha, crearLista(pelota, listaVacía))
```

Sin embargo, para una tienda real, esta lista de seguro contendrá muchos más elementos, y la lista crecerá y disminuirá a lo largo del tiempo. Lo cierto es que en ningún caso podremos decir por adelantado cuántos elementos distintos contendrá la lista. Luego, si quisiéramos desarrollar una función que consuma tales listas, no podremos simplemente decir que la lista de entrada tendrá uno, dos o tres elementos.

En otras palabras, necesitamos una definición de datos que precisamente describa la clase de listas que contenga una cantidad arbitraria de elementos (por ejemplo, strings). Desafortunadamente, hasta ahora sólo hemos visto definiciones de datos que tienen un tamaño fijo, que tienen un número determinado de componentes, o listas con una cantidad delimitada de elementos.

Note que todos los ejemplos que hemos desarrollado hasta ahora siguen un patrón: *comenzamos con una lista vacía, y empezamos a encadenar elementos uno a continuación del otro*. Notemos que podemos abstraer esta idea y plantear la siguiente definición de datos:

Una *lista de strings* es:

1. una lista vacía, `listaVacía`, o bien
2. `crearLista(X, Y)`, donde `X` es un string, y `Y` es una *lista de string*.

Como ya mencionamos anteriormente, nos debería llamar la atención que esta estructura se define *en términos* de sí misma en el segundo ítem. Esto es, para definir una lista, usamos como elemento constituyente otra lista (tal como una cadena se forma al encadenar eslabones entre sí). Estas definiciones se llaman *definiciones recursivas*.

9.3 Procesar listas de largo arbitrario

Tal como vimos anteriormente, en el caso de una tienda nos gustaría poder contar con un inventario de todo el stock de elementos a la venta. En particular, nos interesa mantener este registro en un computador para que un empleado pueda, por ejemplo, verificar si un juguete está o no disponible. Supongamos, pues, que existe la función `hayPelotas` que devuelve `True` si en la tienda quedan pelotas, y `False` en caso contrario. ¿Cómo podríamos implementar esta función?

Dado que ya tenemos una descripción rigurosa de qué es lo que se espera de esta función, sigamos la receta de diseño para escribir su contrato, encabezado y propósito:

```
1 # hayPelotas: lista(str) -> bool
2 # determinar si el string "pelota" esta en la lista unaLista
3 def hayPelotas(unaLista):
4     ...
```

Siguiendo la receta de diseño, debemos ilustrar el funcionamiento de la función a través de unos ejemplos. Primero, debemos determinar cuál es la entrada más simple que puede recibir la función: la lista vacía (`listaVacía`). En este caso, dado que la lista no contiene ningún elemento, es natural que el string `"pelota"` no esté, y por tanto, la salida debe ser `False`:

```
1 >>> hayPelotas(listaVacía)
2 -> False
```


Luego, consideremos listas de un solo elemento:

```
1 >>> hayPelotas(crearLista("pelota", listaVacia))
2     -> True
3 >>> hayPelotas(crearLista("muneca", listaVacia))
4     -> False
```

Y finalmente, veamos el funcionamiento de la función en listas más generales (con más de un elemento):

```
1 >>> hayPelotas(crearLista("arco", crearLista("flecha", \
2               crearLista("muneca", listaVacia))))
3     -> False
4 >>> hayPelotas(crearLista("soldadito", crearLista("pelota", \
5               crearLista("oso", listaVacia))))
6     -> True
```

El paso siguiente es diseñar la plantilla de la función que se corresponda con la definición del tipo de datos. En este caso, dado que la definición de una lista de strings tiene dos cláusulas (lista vacía o una lista de strings), la plantilla se debe escribir usando un bloque condicional `if`:

```
1 # hayPelotas: lista(str) -> bool
2 # determinar si el string "pelota" esta en la lista unaLista
3 # def hayPelotas(unaLista):
4 #     if vacia(unaLista):
5 #         ...
6 #     else:
7 #         ... cabeza(unaLista)
8 #         ... cola(unaLista) ...
```

En este caso, primero preguntamos si la lista está vacía (usando la función `vacia` provista por el módulo `lista.py`). Si no, podemos acceder a la `cabeza` y `cola` de la misma.

Ahora que disponemos de la plantilla para la función, podemos escribir el cuerpo de la misma. Para ello, abordaremos separadamente cada una de las dos ramas de la condición que se definen en la plantilla:

1. Si la lista está vacía, debemos retornar el valor `False` (pues el string no puede estar en una lista vacía).
2. Si no, debemos preguntar si la cabeza corresponde al string buscado. En este caso, surgen dos alternativas:
 - (a) El string efectivamente corresponde a la cabeza: entonces retornamos el valor `True`.
 - (b) Si no, debemos buscar recursivamente dentro de la lista que corresponde a la cola de la lista inicial.

Luego, la definición completa del cuerpo de la función es:

```
1 def hayPelotas(unaLista):
2     if vacia(unaLista):
3         return False
```

```

4         else:
5             if cabeza(unalista) == "pelota":
6                 return True
7             else:
8                 return hayPelotas(cola(unalista))

```

9.3.1 Receta de diseño para funciones con definiciones de datos recursivas

Las definiciones de datos recursivas pueden parecer mucho más complejas que aquellas para datos compuestos, pero como hemos visto en los ejemplos anteriores la receta de diseño que conocemos sigue funcionando. Repasemos la nueva receta de diseño para definiciones de datos recursivas, que generaliza aquella para datos compuestos. La partes nuevas de la receta se preocupan de descubrir cuándo se necesita una definición de datos recursiva, cómo generar una plantilla y como definir el cuerpo de la función:

- **Diseño y análisis de datos:** Si el enunciado de un problema involucra información compuesta de tamaño arbitrario, se necesita una definición de datos recursiva. Hasta ahora, sólo hemos visto listas, pero veremos otras definiciones más adelante. Para que una definición de datos recursiva sea válida, debe satisfacer dos condiciones. Primero, debe contener al menos dos cláusulas. Segundo, al menos una de ellas *no debe referirse* de vuelta a la definición.

Ejemplo: una lista de símbolos contiene dos cláusulas, ya que puede ser

1. una lista vacía, o
2. una lista `crearLista(s, lista(símbolos))`, donde `s` es un símbolo y `lista(símbolos)` es una lista de símbolos.

- **Plantilla:** Se formula con expresiones condicionales. Debe haber una condición por cada cláusula en la definición de datos recursiva, escribiendo expresiones adecuadas en todas las condiciones que procesen datos compuestos.

Ejemplo: la plantilla para una función que procesa una lista de símbolos sería

```

1  # def procesarLista(unalista):
2  #     if vacia(unalista):
3  #         ...
4  #     else:
5  #         ... cabeza(unalista)
6  #         ... procesarLista(cola(unalista)) ...

```

- **Cuerpo de la función:** Se empieza por los casos base (aquellos que no tienen llamados recursivos). Luego, se continúa con los casos recursivos. Debemos recordar cuáles de las expresiones en la plantilla se calculan, y suponemos que para los llamados recursivos la función retorna el valor esperado (hipótesis de inducción). Finalmente, se combinan los valores obtenidos de los llamados recursivos dependiendo del problema (sumarlos, calcular el mínimo, calcular el máximo, etc.).

Ejemplo: supongamos que queremos implementar la función `cuantos`, que calcula cuántos símbolos contiene la lista de símbolos. Siguiendo la receta, tenemos que para el caso base (lista vacía) la respuesta es 0. Para el caso recursivo, tenemos que la lista contiene un símbolo (la cabeza) más los símbolos que contenga el resto de la lista, es decir, basta con sumar 1 al resultado del llamado recursivo para obtener el valor final.

```

1 def cuantos(unaLista):
2     if vacia(unaLista):
3         return 0
4     else:
5         return 1 + cuantos(cola(unaLista))

```

- Combinar valores: El paso de combinar valores puede consistir en una expresión a evaluar (como en el ejemplo anterior), o puede requerir preguntar algo sobre el primer objeto en la lista (en cuyo caso, puede ser necesaria una condición `if` anidada), o puede requerir definir funciones auxiliares (por ejemplo, si se quiere calcular el mínimo de la lista).

9.4 Funciones que producen listas

Recordemos la función `sueldo` que definimos en la Sección 2.2:

```

1 # sueldo: int -> int
2 # calcular el sueldo de un trabajador
3 # (a $4.500 la hora) que ha trabajado h horas
4 def sueldo(h):
5     return 4500 * h

```

Esta función recibe como parámetro el número de horas trabajadas por un empleado, y produce su sueldo semanal. Por simplicidad, supondremos que todos los empleados ganan lo mismo por hora, es decir, \$4.500. Sin embargo, una empresa no está necesariamente interesada en una función como `sueldo`, que calcula el sueldo de un solo empleado, sino más bien en una función que calcule el sueldo total de todos sus empleados (sobre todo si hay muchos).

Llamemos a esta función `listaSueldos`, tal que recibe una lista de cuántas horas los empleados de la compañía han trabajado, y devuelva una lista de los sueldos semanales por cada uno de ellos. Es claro que estas dos listas se pueden representar por *listas de enteros*. Dado que ya disponemos de una definición de datos para la entrada y la salida (modificando levemente la definición que vimos en la sección anterior), podemos comenzar inmediatamente a desarrollar la función.

```

1 # listaSueldos: lista(int) -> lista(int)
2 # crear una lista de sueldos semanales desde
3 # una lista de horas trabajadas (listaHoras)
4 def listaSueldos(listaHoras):
5     ...

```

Luego, como es usual, necesitamos proponer algunos ejemplos de entrada y salida:

```

1 >>> listaSueldos(listaVacia)
2 -> listaVacia
3 >>> listaSueldos(crearLista(10, listaVacia))
4 -> (45000, listaVacia)
5 >>> listaSueldos(crearLista(44, crearLista(10, listaVacia)))
6 -> (198000, (45000, listaVacia))

```

Y, similar al ejemplo anterior, la plantilla a utilizar en la definición de la función es:

```

1 # def listaSueldos(listaHoras):
2 # if (vacía(listaHoras) == True):
3 #     ...
4 # else:
5 #     ... cabeza(listaHoras)
6 #     ... listaSueldos(cola(listaHoras)) ...

```

Ahora que tenemos definida la plantilla podemos proceder a completar el cuerpo de la función. De los ejemplos, tenemos la siguiente primera aproximación:

1. Si `listaHoras` está vacía, entonces tenemos que devolver una lista vacía.
2. Si no, creamos una lista donde la cabeza corresponde a aplicar la función `sueldo` a la cabeza de `listaHoras`, y la encadenamos a la lista que se forma al aplicar recursivamente la función con la cola de `listaHoras`.

Con lo anterior, podemos escribir finalmente el cuerpo de la función:

```

1 # listaSueldos: lista(int) -> lista(int)
2 # crear una lista de sueldos semanales desde una
3 # lista de horas trabajadas (listaHoras)
4 def listaSueldos(listaHoras):
5     if vacía(listaHoras):
6         return listaVacía
7     else:
8         return crearLista(sueldo(cabeza(listaHoras)), \
9                             listaSueldos(cola(listaHoras)))

```

9.5 Listas que contienen estructuras

Volvamos al ejemplo del inventario en la juguetería. En un principio, hicimos la suposición algo ingenua que un inventario consistía únicamente de una lista de elementos. Sin embargo, en registros más formales, se requiere no sólo contar con el listado de los artículos en stock, sino además otras características como el precio, proveedores, código de barras, imágenes, entre otros. De la misma manera, para representar la planilla de sueldos de los empleados de una empresa, por lo general necesitamos más información que la que usamos para modelar una primera versión del problema.

Afortunadamente, recordemos que los elementos de una lista no tienen por qué ser únicamente atómicos (es decir, números, strings o valores lógicos). En efecto, podemos diseñar estructuras y agregarlas sin problemas a cualquier lista que definamos. Así pues, intentemos ir un paso más allá y hagamos el inventario de la juguetería un poco más realista. Comenzaremos entonces por diseñar la estructura que va a representar a la definición de un nuevo tipo de datos específico para este problema: un registro de inventario.

Definiremos un *registro* como una estructura compuesta de un campo de tipo texto para almacenar el nombre del producto, y de un campo de tipo numérico para almacenar el valor de dicho producto. Así pues, diseñemos la estructura:

```

1 # registro: producto(str) precio(int)
2 import estructura
3 estructura.crear("registro", "producto precio")

```

Más aún, podemos definir una *colección de registros* para almacenar toda la información que disponemos. A esto nos referiremos en este problema como *inventario*:

```
1 # inventario: [registro]*
2 # inventario es una lista de registros de largo indeterminado
```

Es decir, un *inventario* está compuesto de:

1. Una lista vacía: `listaVacía`, o bien
2. Una lista que contiene un registro, encadenada al inventario: `crearLista(registro, inventario)`.

De la definición anterior, está claro que la forma más simple de *inventario* es la propia lista vacía: `listaVacía`. Si agregamos un registro, debemos entonces encadenar un nuevo eslabón de la lista a la lista vacía: `crearLista(registro("muneca", 2990), listaVacía)`. Notemos que el elemento que estamos agregando a la lista es una estructura de tipo *registro*. Así, para seguir agregando elementos al inventario, basta con crear un nuevo registro, y encadenarlo a la lista anterior:

```
crearLista(registro("robot", 5990), crearLista( registro("muneca", 2990), listaVacía))
```

Ahora, para hacer las cosas más interesantes, podemos implementar una función que calcule la suma total de los precios que están registrados en el inventario (es decir, calcular el valor total de los productos de la juguetería). En primer lugar, debemos definir el contrato, el propósito, ejemplos, y el encabezado de la función:

```
1 # suma: inventario -> int
2 # calcula la suma de todos los precios en unInventario
3 # suma(listaVacía) = 0
4 # suma(crearLista(("muneca", 2990), listaVacía)) = 2990
5 # suma(crearLista(("robot", 5990), \
6 #   (crearLista("muneca", 2990), listaVacía))) = 8980
7 def suma(unInventario):
8     ...
```

Dado que la definición de un inventario es básicamente la misma que para las listas, nos podemos basar en la plantilla de listas para proponer la plantilla para los inventarios. Así:

```
1 # def suma(unInventario):
2 #     if (vacía(unInventario) == True):
3 #         ...
4 #     else:
5 #         ... cabeza(unInventario)
6 #         ... suma(cola(unInventario)) ...
```

Siguiendo la receta de diseño, recordemos que la plantilla sólo refleja la definición de los datos para la entrada, no para sus componentes.

Finalmente, para definir el cuerpo de la función, debemos considerar cada una de las ramas en la condición de manera independiente. En primer lugar, si el inventario está vacío, naturalmente la `suma` total será 0. Si no, debemos tomar el elemento de la cabeza, acceder al campo `precio` del registro, y luego sumarlo al resultado que arroje la llamada recursiva de la función sobre la cola del inventario.

El cuerpo de la función resulta ser:

```

1  # suma: inventario -> int
2  # calcula la suma de todos los precios en unInventario
3  # suma(listaVacia) == 0
4  # suma(lista(registro("muneca", 2990), lista_vacia)) == 2990
5  # suma(lista(registro("robot", 5990), \
6  #     lista(registro("muneca", 2990), lista_vacia))) == 8980
7  def suma(unInventario):
8      if vacia(unInventario):
9          return 0
10     else:
11         item = cabeza(unInventario)
12         return item.precio + suma(cola(unInventario))
13
14  # Tests
15  suma(lista_vacia) == 0
16  suma(lista(registro("muneca", 2990), lista_vacia)) == 2990
17  suma(lista( registro("robot", 5990), lista( registro("muneca", 2990),
18  listaVacia))) == 8980

```

9.6 Modulo de listas

¿Como será implementar un módulo tan útil como el de las listas? Utilizando estructuras de datos, la respuesta es simple:

Contenido del archivo lista.py

```

1  import estructura
2
3  # Diseno de la estructura
4  # lista : valor (any = cualquier tipo) siguiente (lista)
5  estructura.crear("lista", "valor siguiente")
6
7  # identificador para listas vacias
8  listaVacia = None
9
10 # crearLista: any lista -> lista
11 # devuelve una lista cuya cabeza es valor
12 # y la cola es resto
13 def crearLista(valor, resto):
14     return lista(valor, resto)
15
16 # cabeza: lista -> any
17 # devuelve la cabeza de una lista (un valor)
18 def cabeza(lista):
19     return lista.valor
20
21 # cola: lista -> lista

```

Contenido del archivo lista.py (cont)

```

22 # devuelve la cola de una lista (una lista)
23 def cola(lista):
24     return lista.siguiente
25
26 # vacia: lista -> bool
27 # devuelve True si la lista esta vacia
28 def vacia(lista):
29     return lista == listaVacia
30
31
32 # Tests
33
34 test_lista = lista(1, lista(2, lista(3, listaVacia)))
35
36 assert cabeza(test_lista) == 1
37 assert cabeza(cola(test_lista)) == 2
38 assert cabeza(cola(cola(test_lista))) == 3
39 assert cola(cola(test_lista)) == lista(3, listaVacia)
40
41 assert vacia(listaVacia)
42 assert not vacia(test_lista)
43 assert vacia(cola(cola(cola(test_lista))))

```

9.7 Otras definiciones de datos recursivas

Hasta ahora hemos visto solamente tipos de datos compuestos que requieren una única autorreferencia (como es el caso de las listas). Sin embargo, a menudo nos podemos topar con la necesidad de contar con definiciones más complejas, que requieran de más de una referencia recursiva. Luego, necesitaremos contar con un protocolo para poder definir estas nuevas estructuras, a partir de descripciones informales proporcionadas por el problema a resolver.

Veamos un ejemplo práctico. Típicamente, los médicos utilizan árboles genealógicos para investigar patrones de ocurrencia hereditarios en una familia. Por ejemplo, un árbol genealógico podría servir para ver el seguimiento en los colores de ojos en una familia. ¿Cómo podríamos representar esta estructura en el computador?

Una forma de mantener este árbol genealógico puede ser agregar un nodo cada vez que un niño nace. Desde este nodo, podemos dibujar conexiones al nodo del padre y al nodo de la madre, lo que nos indica que estos nodos están relacionados entre sí. Naturalmente, no dibujamos conexiones para los nodos que no conocemos sus ancestros o descendientes. Más aún, podemos agregar a cada nodo más información relacionada con la persona en cuestión: su nombre, color de ojos, color de pelo, etc.

El ejemplo de la Figura 9.1 nos muestra un árbol genealógico de ancestros de una familia. Esto es, se indica la relación entre los nodos desde los hijos hacia sus padres. Así, Andrés es el hijo de Carlos y Beatriz, tiene ojos pardos y nació en 1950. De manera similar, Gustavo nació en 1996, tiene ojos pardos y es hijo de Eva y Federico. Para representar a un nodo en el árbol combinamos en un único dato compuesto: el nombre, el año de nacimiento, el color de ojos, quién es el padre y quién es la

madre.

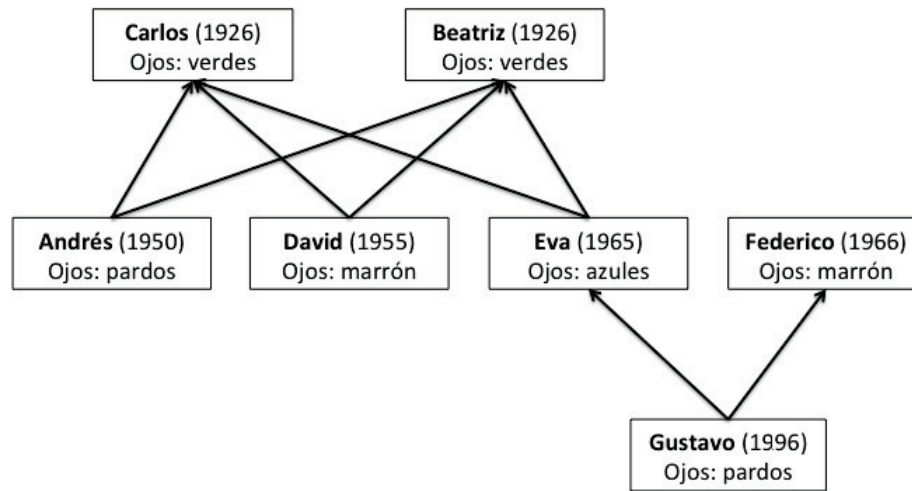


Figura 9.1: Un árbol genealógico de ancestros

Esto sugiere que debemos diseñar una nueva estructura:

```

1 # hijo: padre(hijo) madre(hijo) nombre(str) nacimiento(int) ojos(str)
2 estructura.crear("hijo", "padre madre nombre nacimiento ojos")

```

En otras palabras, un *hijo* se construye a partir de cinco elementos, donde:

1. padre y madre son: (a) vacíos (*listaVacia*), o bien, (b) otro *hijo*; nombre y ojos son strings; nacimiento es un entero.

Esta definición es especial en dos puntos. Primero, es una definición de datos recursiva que involucra estructuras. Segundo, la definición de datos tiene una sola cláusula y menciona dos alternativas para el primer y segundo componente. Esto viola la receta de diseño vista en este capítulo para definiciones de datos recursivas.

Sin embargo, notemos que podemos resolver este problema definiendo simplemente una colección de nodos en un árbol genealógico (*nodoAG*) de la siguiente manera:

```

1 # nodoAG: padre(nodoAG) madre(nodoAG) nombre(str) nacimiento(int) ojos(str)

```

donde la estructura puede ser:

1. vacía (*nodoAGVacio*), o bien
2. otro *nodoAG*

Notemos que, a diferencia del caso anterior, un *nodoAG* no especifica explícitamente cuáles son los tipos de cada una de sus componentes, sino que es recursiva (un *nodoAG* se define en términos de sí mismo). Así, esta nueva definición sí satisface la convención que impusimos de tener al menos dos cláusulas: una no recursiva y una recursiva.

Veamos cómo podemos crear el árbol del ejemplo anterior. Primero, tenemos que crear cada uno de los nodos:

- Carlos: `nodoAG(nodoAGVacio, nodoAGVacio, "Carlos", 1926, "verdes")`
- Beatriz: `nodoAG(nodoAGVacio, nodoAGVacio, "Beatriz", 1926, "verdes")`
- Federico: `nodoAG(nodoAGVacio, nodoAGVacio, "Federico", 1966, "marron")`

Por otro lado, tenemos que definir los nodos más profundos en términos de otros nodos que hay que referenciar en el camino. Esto puede resultar bastante engorroso, por lo que lo más sencillo es introducir una definición de variable por cada nodo, y luego utilizar esta variable para referirnos a los distintos componentes del árbol.

Luego, la definición completa del árbol la podemos transcribir como sigue:

```

1 # Primera generacion:
2 carlos = nodoAG(nodoAGVacio, nodoAGVacio, "Carlos", 1926, "verdes")
3 beatriz = nodoAG(nodoAGVacio, nodoAGVacio, "Beatriz", 1926, "verdes")
4
5 # Segunda generacion:
6 andres = nodoAG(carlos, beatriz, "Andres", 1950, "pardos")
7 david = nodoAG(carlos, beatriz, "David", 1955, "marron")
8 eva = nodoAG(carlos, beatriz, "Eva", 1965, "azules")
9 federico = nodoAG(nodoAGVacio, nodoAGVacio, "Federico", 1966, "marron")
10
11 # Tercera generacion:
12 gustavo = nodoAG(federico, eva, "Gustavo", 1996, "pardos")

```

Ahora que ya contamos con una descripción de cómo podemos manipular árboles genealógicos, nos gustaría poder definir funciones sobre ellos. Consideremos de manera genérica una función del tipo:

```

1 # funcionAG: nodoAG -> ???
2 def funcionAG(unAG):

```

Notemos que deberíamos ser capaces de definir la plantilla de la función, aun sin saber qué es lo que debería producir como resultado. Recordemos que, dado que la definición de la estructura para los `nodoArbol` tiene dos cláusulas, la plantilla debe ser entonces una condición con dos ramas, una para cada cláusula. La primera tiene que verificar el caso `nodoAGVacio` (notar que es necesario implementar la función `vacio`), mientras que la segunda tiene que ocuparse de las componentes de `nodoAG`:

```

1 # unaFuncion: nodoAG -> ???
2 # def unaFuncion(unAG):
3 #     if vacio(nodoAG):
4 #         ...
5 #     else:
6 #         ... unaFuncion(unAG.padre) ...
7 #         ... unaFuncion(unAG.madre) ...
8 #         ... unAG.nombre ...
9 #         ... unAG.nacimiento ...
10 #         ... unAG.ojos ...

```

Veamos ahora un ejemplo concreto. Diseñemos una función que determine si alguien en la familia tiene ojos azules. Como ya debería resultar natural, sigamos la receta de diseño:

```
1 # ancestroOjosAzules: nodoAG -> bool
2 # determina si nodoAG tiene algun ancestro con color de ojos azul
3 def ancestroOjosAzules(unAG):
4     ...
```

Ahora tenemos que desarrollar algunos ejemplos. Consideremos el nodo de Carlos: no tiene ojos azules y como no tienen ningún ancestro (conocido) en la familia, el árbol representado por este nodo no tiene una persona con ojos azules. Luego, `ancestroOjosAzules(carlos)` se evalúa a `False`. Por otro lado, el árbol genealógico representado por Gustavo tiene una referencia al nodo de Eva, quien sí tiene ojos azules. Así, `ancestroOjosAzules(gustavo)` se evalúa a `True`.

La plantilla de la función es similar a la que vimos anteriormente, salvo que ahora nos referimos específicamente a la función `ancestroOjosAzules`. Como siempre, usaremos la plantilla para guiar el diseño del cuerpo de la función:

1. En primer lugar, si `unAG` está vacío, entonces naturalmente nadie puede tener los ojos azules en ese árbol (pues no hay nadie), y la evaluación da `False`.
2. Por otro lado, la segunda cláusula de la plantilla tiene varias expresiones, que debemos revisar una a una:
 - (a) `ancestroOjosAzules(unAG.padre)` verifica si alguien en el árbol del padre del nodo tiene ojos azules;
 - (b) `ancestroOjosAzules(unAG.madre)` verifica si alguien en el árbol de la madre del nodo tiene ojos azules;
 - (c) `unAG.nombre` recupera el nombre del nodo;
 - (d) `unAG.nacimiento` recupera el año de nacimiento del nodo;
 - (e) `unAG.ojos` recupera el color de ojos del nodo;

Nuestra misión es ahora utilizar apropiadamente estos valores. Claramente, si el nodo tiene el string "azules" en el campo `ojos`, entonces el resultado de la función debe ser `True`. En caso contrario, la función debe producir `True` sólo si en el árbol del `padre` o de la `madre` alguien tiene los ojos azules. Este último caso nos indica que debemos verificar *recursivamente* sobre los componentes del árbol para determinar el color de ojos.

Finalmente, dada la discusión anterior, el cuerpo de la función resulta como sigue:

```
1 # ancestroOjosAzules: nodoAG -> bool
2 # determina si nodoArbol tiene algun ancestro con color de ojos azul
3 def ancestroOjosAzules(unAG):
4     if vacio(unAG):
5         return False
6     else:
7         if unAG.ojos == "azules":
8             return True
9         if ancestroOjosAzules(unAG.padre):
10            return True
11        if ancestroOjosAzules(unAG.madre):
```

```

12         return True
13         # si se llega este punto, es porque
14         # ningun ancestro tenia ojos azules
15         return False
16 #Tests
17 assert not ancestroOjosAzules(carlos)
18 assert not ancestroOjosAzules(beatriz)
19 assert ancestroOjosAzules(eva)
20 assert not ancestroOjosAzules(david)
21 assert ancestroOjosAzules(gustavo)

```

9.8 Definiciones mutuamente recursivas

En el ejemplo de la sección anterior, construimos el árbol genealógico desde la perspectiva de los hijos. Sin embargo, es usual construir *árboles genealógicos descendentes*, es decir, indicar quién es descendiente de quién (en lugar de quién es ancestro de quién). La Figura 9.2 muestra el árbol con una perspectiva descendente.

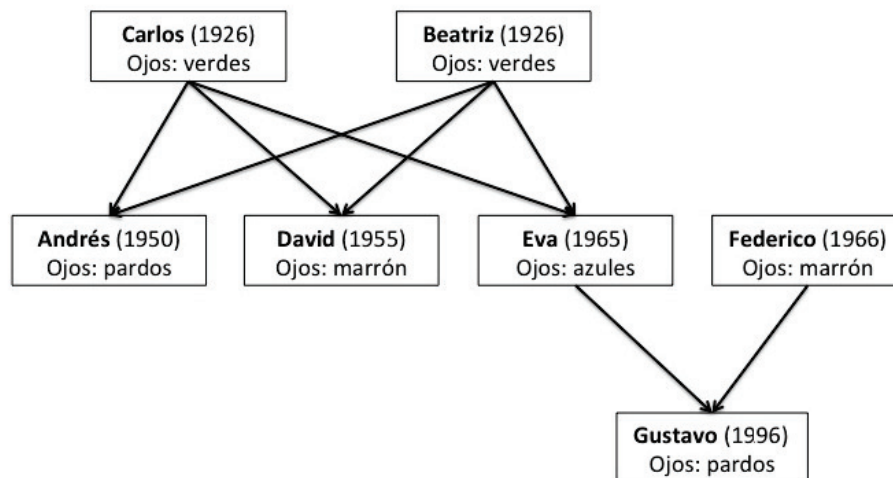


Figura 9.2: Un árbol genealógico de descendientes

No es difícil darse cuenta que para representar este tipo de árboles en el computador necesitamos una estructura de datos diferente al que usamos para el árbol genealógico de ancestros. Esta vez, los nodos van a incluir información sobre los hijos (que a priori no sabemos cuántos son) en lugar de los dos padres.

La estructura se define como sigue:

```

1 # padre: hijos(???) nombre(str) nacimiento(int) ojos(str)

```

Los últimos tres campos de la estructura contienen la misma información básica correspondiente a los nodos de tipo *hijo*. Sin embargo, el primer componente es particular, en cuanto no sabemos cuántos hijos hay (pueden ser 0 o más). Así, la opción natural es definir en este campo una *lista* de nodos de tipo *hijo*. Si una persona no tiene hijos, entonces la lista será *listaVacía*. Luego, debemos

definir la estructura que va a representar a esta lista de hijos.

Una `listaHijos` será:

1. Una lista vacía `listaVacia`, o bien,
2. Una lista de hijos: `crearLista(padre, listaHijos)`.

Notemos que ambas estructuras se refieren mutuamente: para definir a un padre necesitamos una lista de hijos, y para definir a una lista de hijos necesitamos un padre. Así, la única forma que estas dos definiciones hagan sentido es que se introduzcan *simultáneamente*:

```
1 import estructura
2
3 # padre: hijos(listaHijos) nombre(str) nacimiento(int) ojos(str)
4 estructura.crear("padre", "hijos nombre nacimiento ojos")
5
6 # listaHijos: [padre]*
7 # listaHijos es una lista de padres de largo indeterminado
```

Traduzcamos ahora el árbol descendente que presentamos más arriba. Naturalmente, antes de poder crear una estructura de tipo `padre` debemos definir todos los nodos que van a representar a los hijos. Al igual que en la sección anterior, esta definición puede volverse rápidamente muy engorrosa, por lo que lo mejor es introducir variables para llamar a las distintas referencias.

Veamos la generación más joven: tenemos que Gustavo es hijo de Federico y de Eva. Luego, lo primero que debemos hacer es crear un nodo para Gustavo:

```
gustavo = padre(listaVacia, "Gustavo", 1996, "pardos")
```

Creemos ahora el nodo que representa a Federico: tiene como único hijo a Gustavo (que ya definimos previamente). Luego:

```
federico = padre(crearLista(gustavo, listaVacia), "Federico", 1966, "marron")
```

Notemos que en este caso utilizamos la función `crearLista` provista por el módulo `lista.py`, que precisamente permite crear una lista a partir de los valores que recibe como parámetro (en este caso, una estructura de tipo `listaHijos`).

Luego, la traducción completa del árbol es:

```
1 # Nietos:
2 gustavo = padre(listaVacia, "Gustavo", 1996, "pardos")
3 # Padres:
4 hijosFedericoEva = crearLista(gustavo, listaVacia)
5 federico = padre(hijosFedericoEva, "Federico", 1966, "marron")
6 eva = padre(hijosFedericoEva, "Eva", 1965, "azules")
7 david = padre(listaVacia, "David", 1955, "marron")
8 andres = padre(listaVacia, "Andres", 1950, "pardos")
9 # Abuelos:
10 hijosBeatrizCarlos =
    crearLista(andres, crearLista(david, crearLista(eva, listaVacia)))
```

```

11 beatriz = padre(hijosBeatrizCarlos, "Beatriz", 1926, "verdes")
12 carlos = padre(hijosBeatrizCarlos, "Carlos", 1926, "verdes")

```

Veamos ahora el desarrollo de la función `descendienteOjosAzules`, que verifica dentro del árbol si hay algún descendiente que tenga los ojos de color azul. Naturalmente, recibe como parámetro una estructura de tipo `padre` y devuelve un valor de tipo booleano.

```

1 # descendienteOjosAzules: padre -> bool
2 # determina si unPadre o cualquiera de sus
3 # descendientes tiene color de ojos azul
4 def descendienteOjosAzules(unPadre):
5     ...

```

Formulemos directamente tres tests para probar nuestra función:

```

1 # Tests
2 assert not descendienteOjosAzules(gustavo)
3 assert descendienteOjosAzules(eva)
4 assert descendienteOjosAzules(beatriz)

```

Veamos cómo definir la plantilla. Dado que la estructura `padre` tiene cuatro componentes, entonces debemos desagregar cuatro expresiones para poner en la plantilla. Además, como la función no recibe más parámetros, los selectores de la plantilla serán precisamente las cuatro expresiones que se desprenden de las componentes de la estructura:

```

1 # def unaFuncion(unPadre):
2 # ... unPadre.listaHijos ...
3 # ... unPadre.nombre ...
4 # ... unPadre.nacimiento ...
5 # ... unPadre.ojos ...

```

Comencemos ahora a escribir el cuerpo de la función. De la plantilla, notamos que podemos recuperar el color de ojos del padre. Luego, basta con preguntar si esta información es igual a `"azules"` para determinar si el padre tiene los ojos azules:

```

1 def descendienteOjosAzules(unPadre):
2     if unPadre.ojos == "azules":
3         return True
4     else:
5         # ... unPadre.hijos ...
6         # ... unPadre.nombre ...
7         # ... unPadre.nacimiento ...

```

Dado que ni el campo `nombre` ni el campo `nacimiento` nos aportan información relevante respecto al color de ojos de una persona, los descartamos. Así, nos queda finalmente un único selector: `listaHijos`, una estructura que extrae la lista de hijos asociada a un padre dado.

Si el color de ojos de `unPadre` no es `"azules"`, entonces debemos buscar en la `listaHijos` si existe algún descendiente con ojos azules. Siguiendo nuestra guía para diseñar funciones complejas, lo mejor es definir una *función auxiliar* que se encargue de esta tarea y podemos continuar con la resolución del problema. Recordemos que por cada función auxiliar que definamos también tenemos que seguir la receta de diseño:

```

1 # hijoOjosAzules:  listaHijos -> bool
2 # determina si alguna de las estructuras en listaHijos tiene
3 # ojos azules
4 def hijoOjosAzules(unaLista):
5     ...

```

Usando, pues, la función `hijoOjosAzules` podemos completar entonces la definición de nuestra función principal `descendienteOjosAzules`:

```

1 def descendienteOjosAzules(unPadre):
2     if unPadre.ojos == "azules":
3         return True
4     else:
5         return hijoOjosAzules(unPadre.hijos)

```

Ocupémonos ahora de la función `hijoOjosAzules`. Como es natural, veamos primero algunos tests:

```

1 # Tests
2 assert not hijoOjosAzules(hijosFedericoEva) # [Gustavo]
3 assert hijoOjosAzules(hijosBeatrizCarlos) # [Andres, David, Eva]

```

En el ejemplo, Gustavo no tiene ojos azules y no tiene registrado ningún descendiente. Luego, la función debería arrojar el valor `False` para la lista formada por Gustavo. Por otro lado, Eva tiene ojos azules, por lo que el evaluar la función con la lista `[andres, david, eva]` debería arrojar `True`.

Dado que el parámetro que recibe la función es una lista, la plantilla a utilizar es la estándar:

```

1 # def unaFuncion(unaLista):
2 #     if vacia(unaLista) == True:
3 #         ...
4 #     else:
5 #         ... cabeza(unaLista) ...
6 #         ... unaFuncion(cola(unaLista)) ...

```

Consideremos los dos casos:

1. Si `unaLista` está vacía, entonces la respuesta es `False`;
2. Si no, deberíamos preguntar si la cabeza de la lista (de tipo `padre`) tiene ojos azules. Esto propone dos salidas:
 - (a) Si efectivamente la cabeza tiene de la lista tiene ojos azules, retornamos `True`;
 - (b) Si no, preguntamos recursivamente sobre la cola de la lista.

Al respecto, recordemos que la función `descendienteOjosAzules` precisamente verifica si un elemento de tipo `padre` tiene o no los ojos azules. Notemos que la particularidad de esta definición es que ambas funciones son *mutuamente recursivas*, pues las definiciones de estructuras también lo son.

Finalmente, las definiciones de ambas funciones resultan ser:

```

1 # descendienteOjosAzules:  padre -> bool
2 # determina si unPadre o cualquiera de sus
3 # descendientes tiene color de ojos azul

```

```

4 def descendienteOjosAzules(unPadre):
5     if unPadre.ojos == "azules":
6         return True
7     else:
8         return hijoOjosAzules(unPadre.hijos)
9
10 # hijoOjosAzules: listaHijos -> bool
11 # determina si alguna de las estructuras
12 # en listaHijos tiene ojos azules
13 def hijoOjosAzules(unaLista):
14     if vacia(unaLista):
15         return False
16     else:
17         if descendienteOjosAzules(cabeza(unaLista)):
18             return True
19         else:
20             return hijoOjosAzules(cola(unaLista))

```

9.9 Árboles binarios

En computación, no es muy común trabajar con una estructura del tipo **nodoAG** como la que hemos visto de ejemplo en las Sección 9.7. No obstante, una forma particular de árbol que si es muy utilizada es conocida como *árbol binario*. En particular, discutiremos sobre un tipo de árboles denominado *árboles de búsqueda binaria*.

Suponga que se debe implementar una estructura que permita almacenar los datos de personas. En este contexto, un árbol binario es similar a un árbol genealógico, pero ahora utilizaremos la siguiente estructura para sus nodos:

```

1 # nodo: rut(int) nombre(str) izq(nodo) der(nodo)
2 estructura .crear("nodo", "rut nombre izq der")
3
4 # identificador para nodos vacios
5 nodoVacio = None

```

La definición correspondiente de un nodo de árbol binario es:

1. vacío (**nodoVacio**), o bien
2. otro **nodo**, donde **rut** es un número entero (no consideraremos el dígito verificador), **nombre** es un string, y **izq** y **der** son nodos.

Nos va a resultar útil contar con una función que nos permita crear nuevos nodos y otra función que nos diga si un árbol binario es vacío:

```

1 # crearNodo: int str nodo nodo -> nodo
2 # devuelve un nodo ABB con datos rut, nombre
3 # y los nodos izq y der, respectivamente
4 def crearNodo(rut, nombre, izq, der):
5     return nodo(rut, nombre, izq, der)

```

```

6
7 # vacio: nodo -> bool
8 # devuelve True si el ABB esta vacio
9 def vacio(nodo):
10     return nodo == nodoVacio

```

Veamos como crear dos árboles binarios con la definición de nodo propuesta:

```

1 unArbol = crearNodo(15, "Juan", nodoVacio, crearNodo(24, "Ivan",
nodoVacio, nodoVacio))
2 otroArbol = crearNodo(15, "Juan", crearNodo(87, "Hector",
nodoVacio, nodoVacio), nodoVacio)

```

Ahora introduciremos un tipo especial de árbol binario. Un árbol de búsqueda binaria (ABB) es un árbol binario que cumple con una característica de orden con respecto a uno de los valores almacenados (en nuestro ejemplo, el RUT): para todo nodo del árbol se cumple que su valor asociado es mayor que los valores de todos los nodos del lado izquierdo (árbol binario correspondiente a *izq*), y es menor que los valores de todos los nodos del árbol derecho (árbol binario correspondiente a *der*). De acuerdo a esta definición, el árbol `unArbol` sería un ABB, pero el árbol `otroArbol` no lo es. Formalmente, un ABB para nuestro ejemplo se define como:

1. vacío (`nodoVacio`), o bien
2. otro `nodo`, donde se cumple que:
 - (a) *izq* y *der* son ABB
 - (b) todos los números en *izq* son menores que *rut*, y
 - (c) todos los números en *der* son mayores que *rut*.

Note que esta es una definición de datos recursivos válida, ya que contiene al menos dos cláusulas: una no recursiva y una recursiva. Las últimas dos condiciones de la cláusula recursiva son distintas a lo que habíamos visto previamente, ya que agregan restricciones sobre la construcción de un ABB: se debe garantizar que al inspeccionar el árbol *izq* (*der*) todos los valores guardados son menores (mayores) que *rut*.

Para ejemplificar el uso de un ABB, diseñaremos una función que nos permita encontrar en un ABB el nombre asociado a un RUT específico. La plantilla de la función sería la siguiente:

```

1 # encontrarNombre: ABB int -> str
2 # busca el nodo de un ABB cuyo rut es unRUT
3 # y retorna el texto asociado al nombre en dicho nodo
4 # ejemplo:
5 # si ABB = crearNodo(15, "Juan", nodoVacio, crearNodo(24, "Ivan",
nodoVacio, nodoVacio))
6 # entonces encontrarNombre(ABB, 24) devuelve "Ivan"
7 # def encontrarNombre(unNodo, unRUT):
8 #     if vacio(unNodo):
9 #         ...
10 #     else:
11 #         ... encontrarNombre(unNodo.izq) ...
12 #         ... encontrarNombre(unNodo.der) ...
13 #         ... unNodo.rut ...
14 #         ... unNodo.nombre ...

```


Para el caso base, adoptaremos la convención que si el ABB es vacío no hay un nombre asociado al RUT buscado, y la función retornará "" (texto vacío). Para el caso recursivo, vamos a requerir de una expresión condicional: verificamos si el RUT del nodo corresponde al RUT buscado. Si esto es cierto, la función retorna el nombre guardado en el nodo. En caso contrario, la búsqueda en el ABB debe proseguir en forma recursiva, pero la pregunta es, ¿se debe seguir buscando hacia el lado izquierdo o hacia el lado derecho? En este punto es en donde sacamos ventaja que la estructura es un ABB: si el RUT buscado es *menor* que el RUT del nodo, necesariamente debe estar hacia el lado izquierdo del ABB (sino, violaría las restricciones impuestas a un ABB); en caso contrario, si el RUT buscado es *mayor* que el RUT del nodo, necesariamente debe estar hacia el lado derecho del ABB.

Una vez que hemos entendido bien el procedimiento de búsqueda, procedemos a implementarlo:

```
1 # encontrarNombre: ABB int -> str
2 # busca el nodo de un ABB cuyo rut es unRUT
3 # y retorna el texto asociado al nombre en dicho nodo
4 # ejemplo:
5 # si ABB = crearNodo(15, "Juan", nodoVacio, crearNodo(24, "Ivan",
6 # entonces encontrarNombre(ABB, 24) devuelve "Ivan"
7 def encontrarNombre(unNodo, unRUT):
8     if vacio(unNodo):
9         return ""
10    else:
11        if unRUT == unNodo.rut:
12            return unNodo.nombre
13        if unRUT < unNodo.rut:
14            return encontrarNombre(unNodo.izq, unRUT)
15        else: #unRUT > unNodo.rut
16            return encontrarNombre(unNodo.der, unRUT)
17
18 # Tests
19 ABB = crearNodo(15, "Juan", nodoVacio, crearNodo(24, "Ivan",
20            nodoVacio, nodoVacio))
21 assert encontrarNombre(ABB, 24) == "Ivan"
22 assert encontrarNombre(ABB, 50) == ""
```

Capítulo 10

Abstracción Funcional¹

10.1 Similitudes en definiciones

Muchas de las definiciones que creamos, ya sea de datos o de funciones, son parecidas entre sí. Por ejemplo, la definición de una lista de strings se diferencia de una lista de número solo en dos puntos: el nombre de la clase de los datos (es decir, lista-de-strings y lista-de-números) y las palabras “string” y “número”. De igual manera, una función que busca un string específico en una lista de strings es casi indistinguible de una que busque un número específico en una lista de números.

Las repeticiones son la causa de muchos errores de programa. Por lo tanto, los buenos programadores tratan de evitar las repeticiones lo más posible. Cuando desarrollamos un conjunto de funciones, especialmente funciones derivadas de una misma plantilla, pronto aprendemos a encontrar similitudes. Una vez desarrolladas, es tiempo de revisarlas para eliminar las repeticiones lo más posible. Puesto de otra manera, un conjunto de funciones es como un ensayo o una novela u otro tipo de pieza escrita: el primer borrador es sólo un borrador. Es un sufrimiento para otros el tener que leerlos. Dado que las funciones son leídas por muchas otras personas y porque las funciones reales son modificadas después de leerse, debemos aprender a “editar” funciones.

La eliminación de repeticiones es el paso más importante en el proceso de edición de un programa. En esta sección discutiremos las similitudes en la definición de una función y en la definición de datos, y cómo las podemos evitar.

Similitudes entre funciones

El uso de nuestra receta de diseño determina por completo la plantilla de una función, o la organización básica, desde la definición de los datos para la entrada. De hecho, la plantilla es un modo alternativo de expresar lo que sabemos de nuestros datos de entrada. No es de extrañar que las funciones que consumen el mismo tipo de datos se vean parecidas:

¹Parte de este capítulo fue traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

```

# hayPelotas : lista(str) -> bool
# Determina si lista contiene
# el string pelota
# ejemplo hayPelotas(crearLista('pelota',
listaVacia)) devuelve True
def hayPelotas(unaLista):
    if vacia(unaLista):
        return False
    else:
        if cabeza(unaLista) == "pelota":
            return True
        else:
            return hayPelotas(cola(unaLista))

# hayAutos : lista(str) -> bool
# Determina si lista contiene
# el string auto
# ejemplo hayAutos(crearLista('auto',
listaVacia)) devuelve True
def hayAutos(unaLista):
    if vacia(unaLista):
        return False
    else:
        if cabeza(unaLista) == "auto":
            return True
        else:
            return hayAutos(cola(unaLista))

```

Veamos las dos funciones anteriores: ambas consumen una lista de strings (nombres de juguetes) y buscan un juguete en particular. La función de la izquierda busca una pelota y la de la derecha busca un auto en una lista de strings (`unaLista`). Las dos funciones son casi indistinguibles. Cada una consume una lista de strings; cada cuerpo de la función consiste en una expresión condicional con dos cláusulas. Cada una produce `False` si la entrada es vacía; cada una consiste en una segunda expresión condicional anidada para determinar si el primer ítem es el que se busca. La única diferencia es el string que se usa en la comparación de la expresión condicional anidada: `hayPelotas` usa `"pelota"` y `hayAutos` usa `"auto"`.

Los buenos programadores son demasiado perezosos para definir muchas funciones que están estrechamente relacionadas. En lugar de eso, definen una sola función que puede buscar tanto `"pelota"` como `"auto"` en una lista de juguetes. Esa función más general debe consumir un dato adicional, el string que estamos buscando, pero por lo demás es igual que las dos funciones originales.

```

1 # contiene : str lista(str) -> bool
2 # Determina si lista contiene el string s
3 # ejemplo contiene('auto', crearLista('auto', listaVacia)) retorna True
4 def contiene(s, unaLista):
5     if vacia(unaLista):
6         return False
7     else:
8         if cabeza(unaLista) == s:
9             return True
10        else:
11            return contiene(s, cola(unaLista))

```

Ahora podemos buscar `"pelota"` aplicando la función `contiene` a `"pelota"` y a una lista de strings. Pero `contiene` también funciona para cualquier otro string. Definir una sola versión de la función nos permite solucionar muchos problemas relacionados de una sola vez.

El proceso de combinar dos funciones relacionadas en una sola función se denomina *abstracción funcional*. Definir versiones abstractas de funciones es sumamente beneficioso. El primero de estos beneficios es que la función abstracta puede realizar muchas tareas diferentes. En nuestro ejemplo, `contiene` puede buscar muchos strings diferentes en vez de solamente uno particular.

```

# inferiores: lista(num) num -> lista(num)
# Construye una lista de aquellos numeros
# de unaLista que sean inferiores a n
# ejemplo: inferiores(crearLista(1,
# crearLista(2, listaVacia)), 2)
# devuelve (1, listaVacia)
def inferiores(unaLista, n):
    if vacia(unaLista):
        return listaVacia
    else:
        if cabeza(unaLista) < n:
            return crearLista(cabeza(unaLista),
inferiores(cola(unaLista), n))
        else:
            return inferiores(cola(unaLista), n)

# superiores: lista(num) num -> lista(num)
# Construye una lista de aquellos numeros
# de unaLista que sean superiores a n
# ejemplo: superiores(crearLista(2,
# crearLista(4, listaVacia)), 2)
# devuelve (4, listaVacia)
def superiores(unaLista, n):
    if vacia(unaLista):
        return listaVacia
    else:
        if cabeza(unaLista) > n:
            return crearLista(cabeza(unaLista),
superiores(cola(unaLista), n))
        else:
            return superiores(cola(unaLista), n)

```

En el caso de `hayPelotas` y `hayAutos` la abstracción es sencilla. Sin embargo hay casos más interesantes, como son las dos funciones anteriores. La función de la izquierda consume una lista de números y un número, y produce una lista de todos aquellos números de la lista que son inferiores a ese número; la función de la derecha produce una lista con todos aquellos números que están por encima de ese número.

La diferencia entre ambas funciones es el operador de la comparación. La de la izquierda ocupa `<` y la de la derecha `>`. Siguiendo el primer ejemplo, abstraemos las dos funciones usando un parámetro adicional que indica el operador relacional en `inferiores` y `superiores`:

```

1 def filtro(operador, unaLista, n):
2     if vacia(unaLista):
3         return listaVacia
4     else:
5         if operador(cabeza(unaLista), n) :
6             return crearLista(cabeza(unaLista), filtro(operador,
cola(unaLista), n))
7         else:
8             return filtro(operador, cola(unaLista), n)

```

En Python no es posible pasar operadores como argumentos de una función de manera directa. Luego, para poder crear operadores específicos, y posteriormente darlos como argumento a la función `filtro`, crearemos funciones que los definan. Para nuestro ejemplo podemos crear la función `menorQue`, que toma dos argumentos numéricos, y devuelve `True` si el primer argumento es menor que el segundo y `False` en el caso contrario:

```

1 # menorQue: num num -> bool
2 # devuelve True si el primer argumento es menor que el segundo
3 # y False en el caso contrario
4 # Ejemplo: menorQue(4,2) -> False
5 def menorQue(x,y):
6     return x < y

```

Luego, para aplicar esta nueva función, debemos proporcionar tres argumentos: un operador relacional R que compara dos números, una lista L de números, y un número N . La función entonces extrae todos aquellos elementos i en L para las cuales $R(i, N)$ evalúa `True`. Por ahora omitiremos cómo

escribir contratos para funciones como `filtro`, puesto que no sabemos aún como hacerlo. Discutiremos este problema en las secciones que siguen.

Veamos cómo trabaja `filtro` en un ejemplo. Claramente, si la lista que se entrega como parámetro es `listaVacía`, el resultado también será `listaVacía`, sin importar cuáles sean los otros argumentos:

```
1 >>> filtro(menorQue, listaVacía, 5)
2 listaVacía
```

Ahora veamos un ejemplo un poco más complejo:

```
1 >>> filtro(menorQue, crearLista(4, listaVacía), 5)
```

El resultado debería ser `(4, listaVacía)` porque el único elemento de esta lista es 4 y `menorQue(4, 5)` se evalúa como `True`. El primer paso de esta evaluación está basada en la regla de aplicación:

```
1 filtro(menorQue, crearLista(4, listaVacía), 5)
2 =
3 if vacía(crearLista(4, listaVacía)):
4     return listaVacía
5 else:
6     if menorQue(cabeza(crearLista(4, listaVacía)), 5) :
7         return crearLista(cabeza(crearLista(4, listaVacía)), \
8             filtro(menorQue, cola(crearLista(4, listaVacía)), 5))
9     else:
10        return filtro(menorQue, cola(crearLista(4, listaVacía)), 5)
```

Esto significa que es el cuerpo de `filtro` con todas las ocurrencias de `operador` reemplazadas por `menorQue`, `n` reemplazadas por 5, y `unaLista` reemplazada por `crearLista(4, listaVacía)`. El resto de la evaluación es directa:

```
1 if vacía(crearLista(4, listaVacía)):
2     return listaVacía
3 else:
4     if menorQue(cabeza(crearLista(4, listaVacía)), 5) :
5         return crearLista(cabeza(crearLista(4, listaVacía)), \
6             filtro(menorQue, cola(crearLista(4, listaVacía)), 5))
7     else:
8         return filtro(menorQue, cola(crearLista(4, listaVacía)), 5)
9 =
10 if menorQue(cabeza(crearLista(4, listaVacía)), 5) :
11     return crearLista(cabeza(crearLista(4, listaVacía)), \
12         filtro(menorQue, cola(crearLista(4, listaVacía)), 5))
13 else:
14     return filtro(menorQue, cola(crearLista(4, listaVacía)), 5)
15 =
16 if menorQue(4, 5) :
17     return crearLista(cabeza(crearLista(4, listaVacía)), \
18         filtro(menorQue, cola(crearLista(4, listaVacía)), 5))
19 else:
20     return filtro(menorQue, cola(crearLista(4, listaVacía)), 5)
21 =
```

```

22 if True :
23     return crearLista(cabeza(crearLista(4, listaVacía)), \
24         filtro(menorQue, cola(crearLista(4, listaVacía)), 5))
25 else:
26     return filtro(menorQue, cola(crearLista(4, listaVacía)), 5)
27 = crearLista(4, filtro(menorQue, cola(crearLista(4, listaVacía)), 5))
28 = crearLista(4, filtro(menorQue, listaVacía, 5))
29 = crearLista(4, listaVacía)
30 = (4, listaVacía)

```

Nuestro ejemplo final es la aplicación de `filtro` a una lista de dos elementos:

```

1 filtro(menorQue, crearLista(6, crearLista(4, listaVacía)), 5)
2 = filtro(menorQue, crearLista(4, listaVacía), 5)
3 = crearLista(4, filtro(menorQue, listaVacía, 5))
4 = crearLista(4, listaVacía)

```

El único paso nuevo es el primero. Este dice que `filtro` determina que el primer elemento en la lista no es menor que el número entregado como parámetro de comparación, y que por lo tanto no debe ser agregado al resultado de la recursión.

Repeticiones dentro de una función

La repetición de código no sólo se da entre funciones relacionadas, sino que también puede darse dentro de una misma función. Observemos la siguiente función (notar que el código es un *pseudo-Python*, no funciona si Ud. lo transcribe):

```

1 # mayorLargo: lista(any) lista(any) -> num
2 # Devuelve el largo de la lista mas larga, si ambas son vacías
3 # devuelve -1
4 # Ejemplo: mayorLargo (crearLista(5, listaVacía), listaVacía) -> 1
5 def mayorLargo(x, y):
6     if vacía(x) and vacía(y):
7         return -1
8     elif len lista(x) > len lista(y):
9         return len lista(x)
10    else:
11        return len lista(y)

```

En este ejemplo podemos observar que el largo de ambas listas se calcula dos veces. La primera vez se hace para determinar cuál de las dos listas tiene el largo mayor, y la segunda para obtener el valor que retornará la función. Esto significa que existen repeticiones en nuestro código, lo cual no solo afecta a la definición de nuestra función en términos sintácticos, sino que además hace que ésta sea menos eficiente puesto que tiene que evaluar la misma expresión más de una vez. Esto lo podemos mejorar utilizando una *función auxiliar*, que calcule el máximo entre dos valores, y luego utilizarla en nuestra función original para eliminar las repeticiones observadas:

```

1 # maximo: num num -> num
2 # Devuelve el maximo entre x e y
3 # ejemplo: maximo(4, 2) -> 4
4 def maximo(x, y):

```

```

5  if x > y:
6      return x
7  else:
8      return y
9
10 # listaMasLarga: lista lista -> numero
11 # Devuelve el largo de la lista mas larga, si ambas son vacias
12 # devuelve -1
13 # Ejemplo: listaMasLarga (crearLista(5, listaVacía), listaVacía) -> 1
14 def listaMasLarga(x, y):
15     if vacía(x) and vacía(y):
16         return -1
17     else:
18         return maximo(len lista(x), len lista(y))

```

10.2 Similitudes en definición de datos

Veamos las dos siguientes definiciones de estructuras de datos recursivas:

Una lista-de-números puede ser:

- listaVacía
- crearLista(n, l): donde n es un número y l es una lista de números

Una lista-de-*RI*s puede ser:

- listaVacía
- crearLista(n, l): donde n es un *RI* y l es una lista de *RI*

Ambas definen un tipo de listas. La de la izquierda es la definición de un dato que representa una lista de números; la de la derecha describe una lista de registros de inventario (*RI*), que representaremos con una estructura de dato compuesto. La definición de la estructura está dada como sigue:

```

1  import estructura
2  estructura.crear("ri", "nombre precio")

```

Dada la similitud entre la definición de ambas estructuras de datos recursivas, las funciones que consumen elementos de estas clases también son similares. Miremos el siguiente ejemplo. La función de la izquierda es *inferiores*, la cual filtra números de una lista de números. La función de la derecha es *inf-ri*, que extrae todos aquellos registros de inventario de una lista cuyo precio esté por debajo de un cierto número. Excepto por el nombre de la función, la cual es arbitraria, ambas definiciones difieren solo en un punto: el operador relacional.

```

# inferiores : lista(num) num -> lista(num)
# Construye una lista de aquellos numeros
# de unaLista que sean inferiores a n
# ejemplo: inferiores(crearLista(1,
# crearLista(2, listaVacía)), 2)
# devuelve (1, listaVacía)
def inferiores(unaLista, n):
    if vacía(unaLista):
        return listaVacía
    else:
        if cabeza(unaLista) < n:
            return crearLista(cabeza(unaLista),
inferiores(cola(unaLista), n))
        else:
            return inferiores(cola(unaLista), n)

# inf-ri : lista(RI) num -> lista(RI)
# Construye una lista de aquellos registros
# en unaLista con precio inferior a n
# ej.: inf-ri(crearLista(ri("ri1", 1),
# crearLista(ri("ri2", 2), listaVacía)), 2)
# devuelve (ri("ri1", 1), listaVacía)
def inf-ri(unaLista, n):
    if vacía(unaLista):
        return listaVacía
    else:
        if cabeza(unaLista) <ri n:
            return crearLista(cabeza(unaLista),
inf-ri(cola(unaLista), n))
        else:
            return inf-ri(cola(unaLista), n)

```

Si abstraemos las dos funciones, obviamente obtenemos la función `filtro`. Sin embargo, podemos escribir `inf-ri` en términos de `filtro`:

```

def inf-ri(unaLista, n):
    return filtro(<ri, unaLista, n)

```

No nos debería sorprender encontrar otros usos para `filtro` puesto que ya argumentamos que la abstracción funcional fomenta el reuso de funciones para distintos propósitos. Acá podemos ver que `filtro` no solamente filtra lista de números, sino que cosas arbitrarias. Esto se cumple siempre y cuando podamos definir una función que compare estas cosas arbitrarias con números.

En efecto, todo lo que necesitamos es una función que pueda comparar elementos de una lista con elementos que pasamos a `filtro` como el segundo argumento. Acá presentamos una función que extrae todos los elementos con el mismo nombre de una lista de registros de inventario:

```

1 # encontrar : lista(RI) str -> bool
2 # determina si es que unaListaRI contiene un registro de s
3 # ejemplo: encontrar(crearLista(ri("auto", 100), listaVacía), "auto")
4 # devuelve True
5 def encontrar(unaListaRI, s):
6     return not vacía(filtro(igual-ri?, unaListaRI, s))
7
8 # igual-ri? : RI str -> bool
9 # comparar el nombre de un ri y p
10 # ejemplo: igual-ri?(ri("auto", 100), "auto") devuelve True
11 def igual-ri?(ri, p):
12     return ri.nombre == p

```

10.3 Formalizar la abstracción a partir de ejemplos

En los ejemplos vistos anteriormente, partimos de dos definiciones concretas de funciones, las comparamos, marcamos las diferencias, y realizamos la abstracción. Ahora, formularemos una receta para realizar todos estos pasos.

10.3.1 Comparación

Cuando encontramos dos definiciones de funciones que son casi idénticas, salvo algunas pocas diferencias en los nombres, se comparan las funciones y se marcan las diferencias, encerrándolas en una caja. Si las cajas sólo contienen valores, se puede hacer la abstracción. Veamos un ejemplo:

```
# convertirCF : lista(num) -> lista(num)
# Convierte los valores en unaLista
# de grados C a grados F
# ejemplo: convertirCF(crearLista(0,
# listaVacía)) devuelve como resultado
# (32, listaVacía)
def convertirCF(unaLista):
    if vacía(unaLista):
        return listaVacía
    else:
        return
        crearLista(CaF(cabeza(unaLista)),
        convertirCF(cola(unaLista)))

# nombres : lista(RI) -> lista(str)
# Produce una lista de strings con la lista
# de registros en unaLista
# ejemplo: nombres(crearLista(ri("ri1", 1),
# listaVacía)) devuelve como resultado
# ("ri1 1"), listaVacía)
def nombres(unaLista):
    if vacía(unaLista):
        return listaVacía
    else:
        return
        crearLista(RIaString(cabeza(unaLista)),
        nombres(cola(unaLista)))
```

Ambas funciones aplican una función a cada valor de la lista. Se distinguen en un solo punto: la función que aplican (CaF y RIaString, respectivamente), que están marcadas en cajas. Ambos cajas contienen valores funcionales, por lo que se puede realizar la abstracción.

10.3.2 Abstracción

A continuación, se reemplazan los contenidos de los pares de cajas correspondiente con nuevos identificadores, y se agregan dichos nombres a los parámetros. Por ejemplo, si hay tres pares de cajas se necesitan tres identificadores nuevos. Ambas definiciones deben ser ahora idénticas, excepto por el nombre de la función. Para obtener la abstracción, se reemplazan sistemáticamente los nombres de función por uno nuevo.

Para nuestro ejemplo, se obtiene el siguiente par de funciones:

```
def convertirCF(f, unaLista):
    if vacía(unaLista):
        return listaVacía
    else:
        return
        crearLista(f(cabeza(unaLista)),
        convertirCF(f, cola(unaLista)))

def nombres(f, unaLista):
    if vacía(unaLista):
        return listaVacía
    else:
        return
        crearLista(f(cabeza(unaLista)),
        nombres(f, cola(unaLista)))
```

Note que se reemplazaron los nombres en la cajas con `f`, y se agregó `f` como parámetro de ambas funciones. Ahora se reemplazan `convertirCF` y `nombres` con un nuevo nombre de función, obteniéndose la función abstracta `mapa`:

```
1 def mapa(f, unaLista):
2     if vacía(unaLista):
3         return listaVacía
4     else:
5         return crearLista(f(cabeza(unaLista)), mapa(f, cola(unaLista)))
```

Usamos el nombre `mapa` para la función resultante en nuestro ejemplo, dado que es el nombre tradicional en los lenguajes de programación que tiene esta función específica.

10.3.3 Test

Ahora debemos validar que la nueva función es una abstracción correcta de la funciones originales. La definición misma de abstracción sugiere que se puede validar definiendo las funciones originales en términos de la función abstracta, probando las nuevas versiones con los ejemplos de test originales. En la mayoría de los casos, esto se puede realizar en forma directa. Supongamos que la función abstracta se llama `f-abstracta`, y suponga que una de las funciones originales se llama `f-original` que recibe un argumento, llamémosle `valor`. Si `f-original` difiere de las otras funciones originales en el uso de un valor, entonces se define la siguiente función:

```
1 def f-desde-abstracta(x):
2   return f-abstracta(valor, x)
```

Para cada valor (correcto) `v`, `f-desde-abstracta(v)` debiera producir el mismo resultado que `f-original(v)`.

Para nuestro ejemplo, las nuevas definiciones serían las siguientes:

```
def convertirCF-desde-mapa(unaLista):          def nombres-desde-mapa(unaLista):
    return mapa(CaF, unaLista)                  return mapa(RIaString, unaLista)
```

Para asegurarse que estas dos definiciones son equivalentes a las originales, y de paso mostrar que `mapa` es una abstracción correcta, se aplican estas dos funciones a los ejemplos especificados en los contratos originales de `convertirCF` y `nombres`.

10.3.4 Contrato

Para hacer la abstracción útil, debemos formular un contrato adecuado. Si los valores en las cajas de la abstracción (paso dos de esta receta) son funciones, como en el ejemplo, el contrato requiere definirlos, para lo cual utilizaremos tipos de datos con flechas. Adicionalmente, para obtener un contrato flexible, debemos definir y usar definiciones de datos paramétricas y formular un tipo paramétrico.

Veamos como hacer el contrato para la función `mapa` de nuestro ejemplo. Por una parte, si vemos a `mapa` como una abstracción de `convertirCF`, el contrato podría definirse como:

```
# mapa : (num -> num) lista(num) -> lista(num)
```

Por el contrario, si vemos a `mapa` como una abstracción de `nombres`, el contrato quedaría como sigue:

```
# mapa : (RI -> str) lista(RI) -> lista(str)
```

El primer contrato sería inútil en el segundo caso, y viceversa. Para acomodar ambos casos, es necesario entender qué es lo que hace `mapa` y luego definir el contrato. Mirando a la definición de `mapa`, podemos ver que aplica su primer argumento (una función) a cada elemento de su segundo argumento (una lista). Esto implica que la función debe “consumir” el tipo de dato que la lista contenga. Por ejemplo, sabemos que `f` tiene el contrato:

```
# f : X -> ???
```

si `unaLista` contiene datos de tipo `X`. Más aún, `mapa` crea una lista con el resultado de aplicar `f` a cada elemento. Por lo tanto, si `f` produce datos de tipo `Y` entonces `mapa` produce una lista de datos de tipo `Y`. Al agregar todo esto el contrato queda como sigue:

```
# mapa : (X -> Y) lista(X) -> lista(Y)
```

Este contrato indica que `mapa` produce una lista de datos de tipo `Y` a partir de una lista de datos de tipo `X` y una función de `X` a `Y`, independiente de lo que signifique `X` e `Y`.

Una vez que tenemos la abstracción de dos o más funciones, debemos verificar si hay otros usos para la función abstracta. En muchos casos, una función abstracta es útil en una amplia gama de contextos que van más allá de lo que uno inicialmente había anticipado. Esto hace que las funciones sean mucho más fáciles de leer, entender y mantener. Por ejemplo, ahora podemos usar la función `mapa` cada vez que necesitemos una función que produzca una lista nueva a partir del procesamiento de los elementos de una lista ya existente.

10.3.5 Formulando contratos generales

Para aumentar la utilidad de una función abstracta, se debe formular un contrato que describa su aplicabilidad en los términos más generales posible. Para esto, se requiere realizar una abstracción de los contratos, para lo cual se debe seguir la misma receta usada para abstraer funciones. Se comparan y contrastan los contratos originales, luego se reemplazan las diferencias con variables. Realizar este proceso es complejo y requiere práctica.

Veamos un ejemplo con los contratos de las funciones `convertirCF` y `nombres`:

```
# convertirCF : lista(num) -> lista(num)
# nombres : lista(RI) -> lista(str)
```

Al comparar los contratos, se observa que difieren en dos lugares. A la izquierda de `->` se tiene `lista(num)` y `lista(RI)`, a la derecha se tiene `codigolista(num)` y `lista(str)`. Considere el segundo paso de la receta de abstracción. Los contratos quedan como:

```
# mapa : (num -> num) lista(num) -> lista(num)
# mapa : (RI -> str) lista(RI) -> lista(str)
```

Estos contratos sugieren un patrón: el primer argumento, una función, consume los valores en el segundo argumento, una lista; adicionalmente, los resultados producidos en este proceso conforman la salida, otra lista. Fijándonos en el segundo contrato, si se reemplaza `RI` y `str` con variables, se obtiene un contrato abstracto, que de hecho es el contrato formulado para la función `mapa`:

```
# mapa : (X -> Y) lista(X) -> lista(Y)
```

Es fácil comprobar que, reemplazando `X` con `num` e `Y` con `num`, se obtiene el contrato para la función `convertirCF`. Veamos otro ejemplo con los contratos de las funciones `inferiores` e `inf-ri`:

```
# inferiores : lista(num) num -> lista(num)
# inf-ri : lista(RI) num -> lista(RI)
```

Los contratos se diferencian en dos partes: la lista consumida y la lista producida. Como ya hemos visto, las funciones de la segunda etapa del proceso de abstracción reciben un argumento adicional:

```
# filtro : (num num -> bool) lista(num) num -> lista(num)
# filtro : (num RI -> bool) lista(RI) num -> lista(RI)
```

El argumento añadido es una función, que en el primer caso recibe un `num` y en el segundo un `RI`. Comparando ambos contratos, se observa que `num` y `RI` ocupan la misma posición en el contrato, y podemos entonces reemplazarlos por una variable. Al hacer esto, el contrato, que corresponde al de la función `filtro` queda como:

```
# filtro : (num X -> bool) lista(X) num -> lista(X)
# filtro : (num X -> bool) lista(X) num -> lista(X)
```

Observando los nuevos contratos, vemos que ahora es posible reemplazar `num` por otra variable `Y`, con lo que obtenemos el contrato final:

```
# filtro : (Y X -> bool) lista(X) Y -> lista(X)
```

El resultado del primer argumento debe ser `bool` (no puede ser reemplazado por una variable), dado que es utilizado en una condición. Por lo tanto, éste es el contrato más abstracto posible para la función `filtro`.

Resumiendo, para encontrar contratos generales se requiere comparar los contratos de los ejemplos que tengamos para crear abstracciones. Reemplazando valores distintos en posiciones correspondientes por variables, una a la vez, se logra hacer un contrato más genérico en forma gradual. Para validar que la generalización del contrato es correcta, se debe verificar que el contrato describe correctamente las instancias específicas de las funciones originales.

10.4 Otro ejemplo de función abstracta

Otros problemas relacionados con listas que se pueden abstraer en una única función son los siguientes:

- Sumar/multiplicar todos los valores de una lista.
- Concatenar todas las palabras de una lista.

Estos problemas implican procesar los elementos de la lista para obtener un único valor. Esto se puede abstraer a una función que llamaremos `fold` (“reducir”), que recibe una lista, un valor inicial y una función de dos argumentos, procesa los elementos de la lista y devuelve un único valor. Esta función tiene una pre-condición: la lista debe poseer al menos un valor para poder ser procesada. La función toma el valor inicial que se pasó como parámetro y el primer valor de la lista, se invoca la función de dos argumentos y ésta retorna un valor. Dicho valor se ocupa para procesar el segundo valor de la lista usando la función de dos argumentos, y así sucesivamente hasta que se haya procesado la lista completa. El valor que retorna la última invocación a la función de dos argumentos es el valor que devuelve `fold`.

Por ejemplo, para obtener la suma de los valores de una lista, se puede programar de la siguiente forma utilizando `fold`:

```
1 # Funcion de dos argumentos requerida
2 def sumar(x, y)
3     return x + y
4
5 # sumarValoresLista: lista -> num
6 # suma los valores dentro de la lista y devuelve el resultado
```

```

7 # ejemplo: si unaLista = lista(10, lista(20, lista(30, listaVacía)))
8 # sumarValores(unaLista) devuelve 60
9 def sumarValoresLista(unaLista):
10     return fold(unaLista, 0, sumar)

```

Para multiplicar los valores de una lista, se puede usar el siguiente código:

```

1 # Funcion de dos argumentos requerida
2 def multiplicar(x, y)
3     return x * y
4
5 #s umarValoresLista: lista -> num
6 # multiplica los valores dentro de la lista y devuelve el resultado
7 # ejemplo: si unaLista = lista(5, lista(3, lista(3, listaVacía)))
8 # sumarValores(unaLista) devuelve 45
9 def sumarValoresLista(unaLista):
10     return fold(unaLista, 0, multiplicar)

```

Veremos la implementación de `fold` en la Sección 10.6.

10.5 Funciones anónimas

Para evitar el tener que estar definiendo funciones auxiliares que luego son utilizadas sólo como parámetro de las funciones `filtro`, `mapa`, o `fold`, es posible definir *funciones anónimas*. Estas funciones tienen una declaración muy compacta en el código y son funciones “sin nombre”, ya que están pensadas como funciones que se utilizan *una única vez*.

Para definir funciones anónimas en Python se utiliza la instrucción `lambda`. La sintáxis es la siguiente:

```

1 lambda id1, id2, ...: expresion

```

Los identificadores `id1`, `id`, ... corresponde a los parámetros de la función anónima, y `expresion` es la expresión que evalúa la función (y devuelve el resultado). Por ejemplo, una función anónima que suma dos valores se implementa como:

```

1 lambda x,y: x + y

```

Una función anónima booleana que verifica si un número es menor que 5 se implementa como:

```

1 lambda x: x < 5

```

Recuerde que las funciones anónimas están pensadas como funciones que se utilizan una sola vez y luego se desechan. Si la función debe invocarse más de una vez, debe definirse de la manera usual siguiendo la receta de diseño *y no declarar dos veces la misma función anónima*, ya que esto correspondería a duplicación de código, lo que es una mala práctica de programación.

10.6 Resumen de funciones abstractas para listas

A continuación se muestran las implementaciones de las funciones `filtro`, `mapa` y `fold` en su forma más genérica. Los tests se implementan usando funciones anónimas.

10.6.1 Función `filtro`

Note que el contrato en esta implementación es una versión más general que el visto en la Sección 10.3.5, ya que ahora la función booleana recibe un único parámetro.

```

1  # filtro: (X -> bool) lista(X) -> lista(X)
2  # devuelve lista con todos los valores donde operador devuelve True
3  def filtro(operador, unaLista):
4      if vacia(unaLista):
5          return listaVacia
6      else:
7          if operador(cabeza(unaLista)):
8              return lista(cabeza(unaLista), filtro(operador, cola(unaLista)))
9          else:
10             return filtro(operador, cola(unaLista))
11
12 # Tests
13 valores = lista(6, lista(4, lista(8, listaVacia)))
14 assert filtro(lambda x: x < 5, valores) == lista(4, listaVacia)
15 valores = lista('a', lista('b', lista('c', lista('d', listaVacia))))
16 assert filtro(lambda x: x >= 'b' and x < 'd', valores) == \
17             lista('b', lista('c', listaVacia))

```

10.6.2 Función `mapa`

```

1  # mapa : (X -> Y) lista(X) -> lista(Y)
2  # devuelve lista con funcion aplicada a todos sus elementos
3
4  def mapa(f, unaLista):
5      if vacia(unaLista):
6          return listaVacia
7      else:
8          return lista(f(cabeza(unaLista)), mapa(f, cola(unaLista)))
9
10 # Tests
11 valores = lista(1, lista(2, lista(3, lista(4, listaVacia))))
12 assert mapa(lambda x: 10*x, valores) == \
13             lista(10, lista(20, lista(30, lista(40, listaVacia))))
14 valores = lista("pedro", lista("juan", lista("diego", listaVacia)))
15 assert mapa(lambda x: x.upper(), valores) == \
16             lista("PEDRO", lista("JUAN", lista("DIEGO", listaVacia)))

```

10.6.3 Función `fold`

```

1  from lista import *
2
3  # fold: (X X -> X) X lista(X) -> X
4  # procesa la lista con la funcion f y devuelve un unico valor
5  # el valor init se usa como valor inicial para procesar el primer
6  # valor de la lista y como acumulador para los resultados
7  # parciales
8  # pre-condicion: la lista debe tener al menos un valor
9
10 def fold(f, init, unaLista):
11     if cola(unaLista) == listaVacia: # un solo valor
12         return f(init, cabeza(unaLista))

```

```
13     else:
14         return fold(f, f(init, cabeza(unaLista)), cola(unaLista))
15
16 # Tests
17 valores = lista(1, lista(2, lista(3, lista(4, listaVacía))))
18 assert fold(lambda x, y: x + y, 0, valores) == 10
19 valores = lista("pedro", lista("juan", lista("diego", listaVacía)))
20 assert fold(lambda x, y: x + y, "", valores) == "pedrojuandiego"
```